Machine Learning Zoomcamp

Module 5 — Deploying Machine Learning Models

Study notes with diagrams and code

Based on the DataTalksClub course notes and workshop, `05-deployment`

Condensed and re-organised from
`https://github.com/DataTalksClub/machine-learning-zoomcamp/tree/main/05-deployment`.

The module videos use Flask + Pipenv + AWS Elastic Beanstalk; the workshop replaces them with FastAPI + uv + Fly.io. These notes cover both, side by side.

All outputs were re-run on the Telco churn dataset with pandas 3.0.6, scikit-learn 1.9.1, FastAPI 0.162 and Pydantic 2.14 (see the boxes in §3 for two version gotchas).

Contents

1 Why Deploy? 3

1.1 The plan: remove one dependency on your laptop at a time . . . 3

2 Saving and Loading the Model 3

2.1 Scikit-learn pipelines (workshop) . . . 4

3 From Notebook to Script 4

4 Web Services in One Page 5

5 Serving the Model 5

5.1 Hello world: ping/pong . . . 5

5.2 The churn service . . . 6

5.3 The client . . . 7

5.4 Flask vs FastAPI . . . 7

6 Input Validation with Pydantic (workshop) 7

7 Production Servers 8

8 Virtual Environments: Pipenv and uv 8

9 Docker 9

9.1 The Dockerfile . . . 10

9.2 Build and run . . . 10

10 Deploying to the Cloud 11

11 Summary 12

11.1 Command cheat sheet (workshop stack) 12

A Appendix: Troubleshooting and Explore More **13**

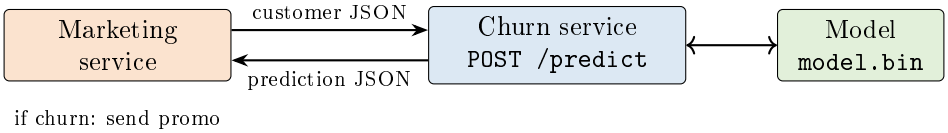
A.1 Symptom → likely cause 13

A.2 Explore more (from unit 5.9) 13

1 Why Deploy?

In Module 3 we trained a model that predicts whether a telecom customer will **churn**. Right now it lives in a notebook: to get a prediction you open Jupyter, run every cell and read the output. That works for us, not for the rest of the company.

The marketing team wants to send a promo email to customers who are likely to leave. Their system needs to ask *automatically, for thousands of customers*: "what is the churn probability of this customer?" The answer is to **deploy** the model: run it on a server behind an **API endpoint**. Other services send customer data in a request and get the prediction back in the response.



1.1 The plan: remove one dependency on your laptop at a time

Each step of the module removes one thing that ties the model to the machine it was trained on:

	Step	Removes the dependency on...	Course → Workshop
1	Save the model to a file	the notebook's memory	pickle
2	Turn the notebook into a script	Jupyter	`nbconvert`, `train.py`
3	Wrap it in a web service	a human running the code	Flask → FastAPI
4	Use a production server	the dev server	gunicorn → uvicorn
5	Isolate the Python packages	system-wide `pip`	Pipenv → uv
6	Containerise	the OS and Python version	Docker
7	Deploy to the cloud	your machine itself	Elastic Beanstalk → Fly.io

2 Saving and Loading the Model

Training produces *two* objects that live only in the notebook process: the fitted `DictVectorizer` and the `LogisticRegression`. Close the notebook and they are gone. Re-training on every request would be slow, and could give a slightly different model each time. So we **train once, save to a file, load in the service**.

Pickle is Python's built-in serialiser: it turns an object into bytes and back.

```
import pickle

with open('model_C=1.0.bin', 'wb') as f_out:       # 'wb' = write binary
    pickle.dump((dv, model), f_out)                # save BOTH as a tuple

with open('model_C=1.0.bin', 'rb') as f_in:        # 'rb' = read binary
    dv, model = pickle.load(f_in)
```

Why the vectorizer must travel with the model

The model only understands the 45 numeric columns that `dv` produced during training (e.g. `contract=month-to-month`, `internetservice=dsl`, ...). `dv` "remembers" which categorical value maps to which column. A new customer has to be encoded with *the same* fitted `dv`; a freshly created one would produce different columns, or none at all. Putting `C=1.0` in the filename records which hyper-parameter produced the file.

2.1 Scikit-learn pipelines (workshop)

Carrying `(dv, model)` around is error-prone. A **pipeline** chains them into one object whose `fit`/`predict_proba` run every step in order:

```
from sklearn.pipeline import make_pipeline

pipeline = make_pipeline(
    DictVectorizer(),
    LogisticRegression(solver='liblinear'),
)
pipeline.fit(train_dicts, y_train)              # list of dicts in, no manual transform
pipeline.predict_proba(customer)[0, 1]          # one dict is fine too

with open('model.bin', 'wb') as f_out:
    pickle.dump(pipeline, f_out)                # one object, one file (~2.6 KB here)
```

Pickle safety and versions

- **Never unpickle a file you do not trust.** Unpickling can execute arbitrary code.
- **Pin the library version.** A pickle stores references to scikit-learn classes, not a frozen copy of them. Loading it with a different scikit-learn version gives an `InconsistentVersionWarning` at best and wrong behaviour at worst. This is one reason §8 (lock files) and §9 (Docker) exist.

3 From Notebook to Script

A server runs scripts, not notebooks. A script is reproducible, can be started by a scheduler or a CI job, and needs no Jupyter.

```
jupyter nbconvert --to=script workshop-uv-fastapi.ipynb
mv workshop-uv-fastapi.py train.py        # then clean it up
```

After cleaning up, the workshop's `train.py` has three functions plus a "main" part:

```
def load_data():          # read CSV, normalise column names/values, fix totalcharges
def train_model(df):      # dicts -> pipeline.fit
def save_model(pipeline, output_file):   # pickle.dump

df = load_data()
pipeline = train_model(df)
save_model(pipeline, 'model.bin')
```

The module's version also does `KFold` cross-validation and prints the AUC for `C=1.0` before training the final model. A separate `predict.py` only *loads* `model.bin`; training and serving are separate programs.

Gotcha 1 (pandas ≥ 3): string columns are no longer `object`

The course code selects text columns with `df.dtypes[df.dtypes == 'object']`. In pandas 3 strings default to the new `str` dtype, so **this selects nothing**. The values are never lowercased, `df.churn == 'yes'` is `False` everywhere, and the target becomes all zeros (churn rate 0.0). It fails silently; scikit-learn's error only shows up later. Fix:

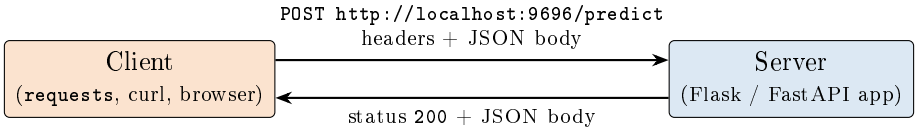
```
for c in df.select_dtypes(include=['object', 'string']).columns:
    df[c] = df[c].str.lower().str.replace(' ', '_')
```

Gotcha 2 (scikit-learn 1.9): sparse `DictVectorizer` + `liblinear`

With scikit-learn 1.9.1, `make_pipeline(DictVectorizer(), LogisticRegression(solver='liblinear'))` fails on `fit` with *"Only sparse matrices with 32-bit integer indices are accepted. Got int64 indices."* Use `DictVectorizer(sparse=False)` (as in the Module 3–4 homework), or another solver. With both fixes: 7043 rows, churn rate 0.265, 45 features.

4 Web Services in One Page

A **web service** lets applications talk over **HTTP**, the protocol your browser uses. A *client* sends a *request* to a *URL*; the *server* sends back a *response*.



A URL such as `http://localhost:9696/predict` has four parts: the protocol (`http`), the host (`localhost`), the port (`9696`) and the route (`/predict`).

Method	Meaning	In this module
`GET`	retrieve something (what a browser sends)	`/ping` health check
`POST`	send data in the body	`/predict` with a customer
`PUT`	like POST, but the request says where data goes	—
`DELETE`	remove something	—

`localhost` vs `0.0.0.0`

`localhost` (`127.0.0.1`) means "this machine". A server *bound* to it only accepts connections from the same machine. Binding to `0.0.0.0` means "listen on every network interface", so requests from outside are accepted too. This matters inside Docker (§9): a container is a separate machine as far as networking goes, so the app **must** bind to `0.0.0.0` or the host cannot reach it.

5 Serving the Model

5.1 Hello world: ping/pong

Flask (module)

```python
from flask import Flask

app = Flask('ping')

@app.route('/ping', methods=['GET'])
def ping():
    return "PONG"

if __name__ == "__main__":
    app.run(debug=True,
            host='0.0.0.0', port=9696)
```

FastAPI (workshop)

```python
from fastapi import FastAPI
import uvicorn

app = FastAPI(title="ping")

@app.get("/ping")
def ping():
    return "PONG"

if __name__ == "__main__":
    uvicorn.run(app,
            host="0.0.0.0", port=9696)
```

- The **decorator** (`@app.route` / `@app.get`) registers the function as the handler for that HTTP method on that **route**.
- `if __name__ == "__main__"` runs only for `python ping.py`, not when another program (gunicorn, uvicorn) *imports* the file.
- Test with `curl localhost:9696/ping` or a browser. With FastAPI, also open `http://localhost:9696/docs`: interactive Swagger docs, generated automatically.

5.2 The churn service

Flask (module)

```python
import pickle
from flask import Flask, request, jsonify

with open('model_C=1.0.bin', 'rb') as f_in:
    dv, model = pickle.load(f_in)

app = Flask('churn')

@app.route('/predict', methods=['POST'])
def predict():
    customer = request.get_json()

    X = dv.transform([customer])
    y_pred = model.predict_proba(X)[0, 1]
    churn = y_pred >= 0.5

    return jsonify({
        'churn_probability': float(y_pred),
        'churn': bool(churn),
    })
```

FastAPI (workshop)

```python
import pickle
from fastapi import FastAPI

app = FastAPI(title="customer-churn-prediction"
    )

with open('model.bin', 'rb') as f_in:
    pipeline = pickle.load(f_in)

def predict_single(customer):
    result = pipeline.predict_proba(customer)
    [0, 1]
    return float(result)

@app.post("/predict")
def predict(customer: Customer) ->
    PredictResponse:
    prob = predict_single(customer.model_dump()
    )
    return PredictResponse(
        churn_probability=prob,
        churn=prob >= 0.5,
    )
```

What both versions have in common:

1. **The model is loaded once, at start-up**, at module level, not inside the handler. Every request reuses the object already in memory.
2. **POST**: the customer arrives as JSON in the request body.
3. `predict_proba(X)[0, 1]`: row 0 (the only customer), column 1 (P(churn), the positive class).
4. **Threshold 0.5** turns the probability into a decision. The client gets both.

Why the casts `float(...)` and `bool(...)`?

JSON only knows native Python types. Checked with NumPy 2.5:

Value	`json.dumps`	Why
`np.float64(0.66)`	works	`np.float64` subclasses Python `float`
`np.float32(0.66)`	`TypeError`	not a `float` subclass
`y_pred >= 0.5` → `np.bool_`	`TypeError`	not a `bool` subclass

So the `bool()` cast on the decision is the one that really matters. Casting both is the safe habit. In FastAPI, the `PredictResponse` model does the conversion for you.

5.3 The client

The browser cannot test `/predict`: typing a URL sends a GET, and the route expects a POST. A small client script uses `requests`:

```
import requests

url = 'http://localhost:9696/predict'
customer = {'gender': 'female', 'seniorcitizen': 0, 'partner': 'yes', ...,
            'contract': 'month-to-month', 'tenure': 1,
            'monthlycharges': 29.85, 'totalcharges': 29.85}

response = requests.post(url, json=customer).json()   # json= serialises + sets header
print(response)
if response['churn']:
    print('customer is likely to churn, send promo')
```

For the workshop customer (a new, month-to-month, electronic-check customer), the pipeline trained on all 7043 rows returns

$$P(\text{churn}) = 0.664 \;\geq\; 0.5 \quad\Longrightarrow\quad \text{churn} = \text{true} \quad\Longrightarrow\quad \text{send promo.}$$

The same customer with 41 months of tenure, a two-year contract and automatic bank transfer gets $P = 0.134$, so no promo. The curl equivalent needs the header explicitly: `curl -X POST localhost:9696/predict -H 'Content-Type: application/json' -d '{...}'`.

5.4 Flask vs FastAPI

	Flask	**FastAPI**
Route	`@app.route('/p', methods=['POST'])`	`@app.post('/p')`
Read body	`request.get_json()` (manual)	typed function argument, parsed for you
Validation	none; you write it	automatic from type hints / Pydantic
Bad input	usually a 500 crash deep in `dv.transform`	422 with a precise error message
Docs	—	`/docs` (Swagger) and `/openapi.json`
Return JSON	`jsonify(dict)`	return a dict or a Pydantic model
Server spec	WSGI (sync) → gunicorn / waitress	ASGI (async capable) → uvicorn

6 Input Validation with Pydantic (workshop)

An untyped handler `def predict(customer)` isn't even read as a JSON body by FastAPI. With `customer: Dict[str, Any]` it is, but anything goes. A **Pydantic model** states exactly which fields and values are allowed:

```
from typing import Literal
from pydantic import BaseModel, ConfigDict, Field

class Customer(BaseModel):
    model_config = ConfigDict(extra="forbid")          # reject unknown fields

    gender: Literal["male", "female"]
    seniorcitizen: Literal[0, 1]
    contract: Literal["month-to-month", "one_year", "two_year"]
    streamingtv: Literal["no", "yes", "no_internet_service"]
    # ... the other 12 categorical fields, same pattern ...
    tenure: int = Field(..., ge=0)                     # ... = required, ge = >=
    monthlycharges: float = Field(..., ge=0.0)
    totalcharges: float = Field(..., ge=0.0)
```

```python
class PredictResponse(BaseModel):
    churn_probability: float
    churn: bool
```

The allowed `Literal` values come straight from `df[c].value_counts()` and the ranges from `df[n].describe()`. Inside the handler, `customer.model_dump()` turns the model back into the plain dict the pipeline expects (`.dict()` still works but is deprecated in Pydantic v2).

Verified behaviour (FastAPI 0.162, Pydantic 2.14):

Request body	Status	Response
valid customer	200	`{"churn_probability": ..., "churn": ...}`
+ `"whatever": 31337`, *no* `extra="forbid"`	200	silently ignored
+ `"whatever": 31337`, with `extra="forbid"`	422	`extra_forbidden`: "Extra inputs are not permitted"
`"streamingtv": "maybe"`	422	`literal_error`: "Input should be 'no', 'yes' or 'no_internet_service"'
`"tenure": -3`	422	`greater_than_equal`: "Input should be greater than or equal to 0"

Why validate at the boundary?

Without validation, `"streamingtv": "maybe"` goes straight into `DictVectorizer`, which *silently ignores* unseen categories. The model returns a confident-looking probability for garbage input. A `422` at the door is far better than a wrong answer from inside.

7 Production Servers

`app.run(debug=True)` starts Flask's **development server** and prints a warning: it is single-threaded, has a debugger that can execute code, and is not built for load. In production a dedicated server *imports* your `app` object and handles the networking, workers and concurrency.

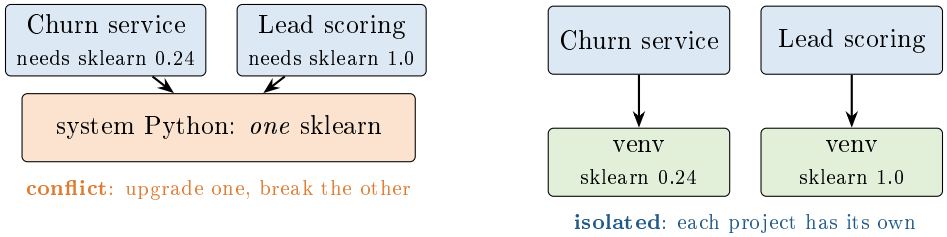
- **WSGI** (Web Server Gateway Interface) is the standard interface between Python servers and *synchronous* apps like Flask.
- **ASGI** is its asynchronous successor, used by FastAPI.

```
gunicorn --bind=0.0.0.0:9696 predict:app                  # Flask, Linux/macOS
waitress-serve --listen=0.0.0.0:9696 predict:app          # Flask, Windows
uvicorn predict:app --host 0.0.0.0 --port 9696 --reload   # FastAPI (--reload: dev only)
```

The argument `predict:app` always means `module:object`: import `predict.py`, use the variable `app` inside it. gunicorn does not run on Windows (it relies on Unix process forking), hence waitress.

8 Virtual Environments: Pipenv and uv

`pip install` puts every package into one shared, system-wide Python. That works until two projects need different versions of the same library:



A **virtual environment** is a private directory of packages (plus a `python` and `pip` that use it) for one project. Many tools manage them: `venv`, conda, poetry, **Pipenv** (module) and **uv** (workshop). The last two do more than isolate: they also **lock** exact versions so the environment can be rebuilt identically on a server.

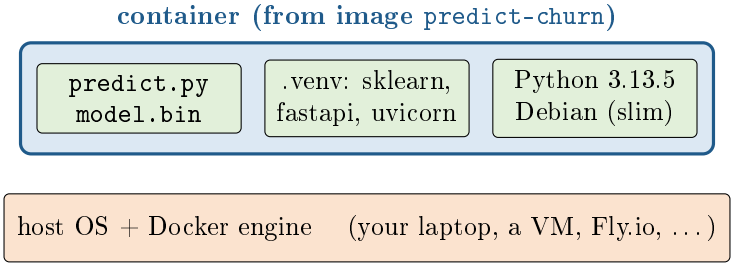
Task	Pipenv	uv
Start a project	(implicit on first install)	`uv init` (creates `pyproject.toml`, `.python-version`)
Add dependencies	`pipenv install numpy scikit-learn flask gunicorn`	`uv add scikit-learn fastapi uvicorn`
Add a dev-only dep	`pipenv install -dev awsebcli`	`uv add -dev requests`
Declared deps	`Pipfile`	`pyproject.toml`
Exact, hashed versions	`Pipfile.lock`	`uv.lock`
Rebuild on new machine	`pipenv install`	`uv sync`
Run a command	`pipenv run gunicorn ...` / `pipenv shell`	`uv run uvicorn ...`
Speed	slow resolver	very fast (Rust)

Declared vs locked

The **declared** file lists what you asked for, often loosely (`fastapi>=0.116.1`, `numpy = "*"`). The **lock** file records what was actually resolved: every package *including transitive dependencies*, with exact versions and hashes. Commit both. The lock file is what turns "works on my machine" into "works on every machine". **Dev dependencies** (`requests` for the test client, `awsebcli`) are needed on your laptop but not in production, and can be left out of the image.

9 Docker

A virtual environment isolates *Python packages* only. The OS, the system libraries and the Python version itself are still whatever the host has. A service tested on Python 3.13 can fail on a server with 3.8. **Docker** packs the *whole* environment into an **image**; a **container** is a running instance of it and behaves the same on any machine with Docker.



9.1 The Dockerfile

An image is built from a **Dockerfile**, a recipe that starts from a **base image** (here an official Python image from Docker Hub) and adds one **layer** per instruction.

```
FROM python:3.13.5-slim-bookworm                          # base: Python on minimal Debian

COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/  # grab the uv binary
WORKDIR /code                                             # cd (creates the dir)
ENV PATH="/code/.venv/bin:$PATH"                          # use the venv's python/uvicorn

COPY "pyproject.toml" "uv.lock" ".python-version" ./    # 1. dependency files only
RUN uv sync --locked                                      # 2. install exactly uv.lock

COPY "predict.py" "model.bin" ./                          # 3. then code + model

EXPOSE 9696                                               # document the port
ENTRYPOINT ["uvicorn", "predict:app", "--host", "0.0.0.0", "--port", "9696"]
```

The module's Pipenv version has the same shape:

```
FROM python:3.8.12-slim
RUN pip install pipenv
WORKDIR /app
COPY ["Pipfile", "Pipfile.lock", "./"]
RUN pipenv install --system --deploy
COPY ["predict.py", "model_C=1.0.bin", "./"]
EXPOSE 9696
ENTRYPOINT ["gunicorn", "--bind=0.0.0.0:9696", "predict:app"]
```

- `--system`: install into the image's Python instead of creating a venv inside the container (the container is already isolated). `--deploy`: fail if `Pipfile.lock` is out of date. `uv sync --locked` is uv's equivalent check.
- **Slim** base images are much smaller, so builds and pulls are faster.
- **ENTRYPOINT** is what runs when the container starts. Without it, `python:*` images just start a Python REPL. The JSON-array ("exec") form runs the command directly, without a shell in between.
- Pin the base image (`3.13.5-slim-bookworm`, not `latest`) to match the Python you developed with (`python -V`).

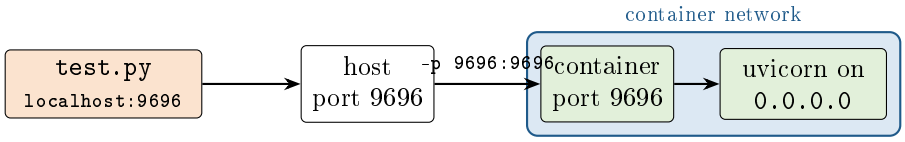
Why copy the dependency files before the code? (layer caching)

Docker caches each layer and rebuilds only from the first instruction whose inputs changed. Dependencies change rarely; `predict.py` changes often. With the order above, editing the code invalidates only step 3 and the slow `uv sync` is reused from cache. With `COPY . .` first, *every* code edit would reinstall all packages. A `.dockerignore` keeps `.venv/` and other local clutter out of the build context.

9.2 Build and run

```
docker build -t predict-churn .                # -t name:tag (default tag "latest"); . = build
    context
docker run -it --rm -p 9696:9696 predict-churn   # then: python test.py  (unchanged URL)
```

Flag	Meaning
`-it`	interactive + terminal: see the logs, stop with Ctrl-C
`-rm`	delete the *container* when it stops (the image stays)
`-p H:C`	publish container port `C` on host port `H`
`-entrypoint=bash`	override the entrypoint, to poke around inside the image



`EXPOSE` only *documents* the port. `-p` is what actually opens it. Forget `-p`, or bind the app to `localhost` inside the container, and the client gets "connection refused".

10 Deploying to the Cloud

Once the service is a Docker image, it can run anywhere that runs containers. The course uses AWS Elastic Beanstalk; the workshop uses Fly.io. Google Cloud Run, AWS App Runner, Azure, Heroku and PythonAnywhere also work (see the community tutorials in the module README).

AWS Elastic Beanstalk (module, optional)

```
pipenv install --dev awsebcli
eb init -p docker -r eu-north-1 churn-serving
eb local run --port 9696    # test locally
    first
eb create churn-serving-env # ~minutes; prints
    URL
# ... point test.py at the EB URL (no port) ...
eb terminate churn-serving-env
```

Fly.io (workshop)

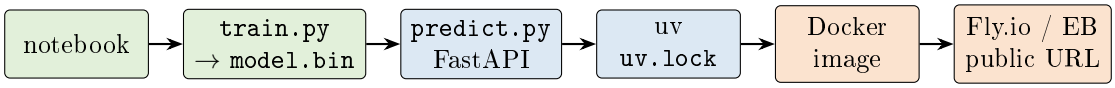
```
curl -L https://fly.io/install.sh | sh
fly auth signup
fly launch --generate-name  # writes fly.toml
fly deploy                  # builds + ships
    image
# Visit https://<app>.fly.dev/predict
fly apps list
fly apps destroy <app-name>
```

- `eb init -p docker`: the platform is Docker, so EB builds and runs our Dockerfile. `-r` picks the region (choose one near your users). Configuration lands in `.elasticbeanstalk/config.yml`.
- **Scaling**: EB puts a load balancer in front and adds container instances when traffic grows (*horizontal scaling*), then removes them when it drops. Fly's `fly.toml` has `auto_stop_machines` / `min_machines_running = 0`: the app scales to zero when idle.
- The public URL uses the standard HTTP(S) port, so it has **no** `:9696`. The platform routes it to the `internal_port = 9696` of the container.

Before you walk away from a cloud deployment

- **It is public.** Anyone with the URL can call it. Real deployments need authentication or network-level access control.
- **It costs money while it runs.** Always `eb terminate` / `fly apps destroy` when you are done, and check the provider's pricing first.
- Configure AWS credentials before using the `eb` CLI.

11 Summary



1. **Save the model** with pickle: vectorizer + model together, or one sklearn `Pipeline`. Only unpickle trusted files, with the same library versions.
2. **Script it**: training (`train.py`) and serving (`predict.py`) are separate programs.
3. **Serve it** over HTTP: `POST /predict`, JSON in, JSON out, model loaded once at start-up, native types in the response.
4. **Validate input** with Pydantic so bad requests get a 422 instead of a silent wrong answer.
5. **Use a production server**: gunicorn/waitress (WSGI, Flask) or uvicorn (ASGI, FastAPI).
6. **Lock dependencies** in a virtual environment: Pipenv or uv.
7. **Containerise** with Docker: dependency files first, then code. Bind to `0.0.0.0`, publish with `-p`.
8. **Deploy**, test with the public URL, then **tear it down**.

11.1 Command cheat sheet (workshop stack)

```
uv init && rm main.py
uv add scikit-learn fastapi uvicorn && uv add --dev requests
uv run python train.py                                    # -> model.bin
uv run uvicorn predict:app --host 0.0.0.0 --port 9696 --reload
uv run python test.py                                     # in another terminal
docker build -t predict-churn . && docker run -it --rm -p 9696:9696 predict-churn
fly launch --generate-name && fly deploy                  # ... fly apps destroy <name>
```

A Appendix: Troubleshooting and Explore More

A.1 Symptom → likely cause

Symptom	Likely cause / fix
Churn rate 0.0, odd errors at `fit`	pandas 3 `str` dtype: `dtypes == 'object'` matches nothing (§3)
"Only sparse matrices with 32-bit integer indices"	sklearn 1.9 + sparse `DictVectorizer` + liblinear → `sparse=False`
`TypeError`/`UnicodeDecodeError` on dump/load	opened with `'r'`/`'w'` instead of `'rb'`/`'wb'`
`InconsistentVersionWarning` on load	sklearn version differs from training → use the lock file
`TypeError: ... not JSON serializable`	returning `np.bool_`/`np.float32` → cast with `bool()`/`float()`
`405 Method Not Allowed` in the browser	browser sends GET to a POST-only route → use `requests`/curl//`docs`
`422 Unprocessable Entity`	Pydantic rejected the body: read `detail[].loc` and `msg`
"Connection refused" to a container	missing `-p 9696:9696`, or app bound to `localhost` not `0.0.0.0`
Port 9696 already in use	an old server/container is still running (`docker ps`, Ctrl-C)
Every code edit reinstalls all packages	`COPY . .` before `RUN uv sync`: reorder the Dockerfile
`gunicorn` fails on Windows	use `waitress-serve`, or run inside Docker/WSL

A.2 Explore more (from unit 5.9)

- Other web frameworks besides Flask, e.g. FastAPI (done in the workshop), BentoML.
- Other environment managers: `venv`, conda, poetry (and uv).
- Other deployment targets: Google Cloud Run, Azure, Heroku, PythonAnywhere, AWS App Runner.
- Next module: decision trees, random forests and gradient boosting, a different model to drop into the exact same serving stack.